



σ_0 : large scale quant foundation models

Aaron Rose

BOLD British Open Ended Discovery Lab

Contents

1	Data	1
2	Pretraining	5
3	Sampling	9

1 Data

1.1 The book

A snapshot of the order book at time t is

$$S_t = \begin{bmatrix} a_1 & u_1 & b_1 & w_1 \\ \vdots & \vdots & \vdots & \vdots \\ a_{10} & u_{10} & b_{10} & w_{10} \end{bmatrix} \in \mathbb{R}^{10 \times 4}, \quad (1)$$

with a_j and b_j the ask and bid price at level j , and u_j and w_j the depth at that price. Prices are integers in units of 10^{-4} dollars, so one cent is 100. Figure 1 shows one snapshot.

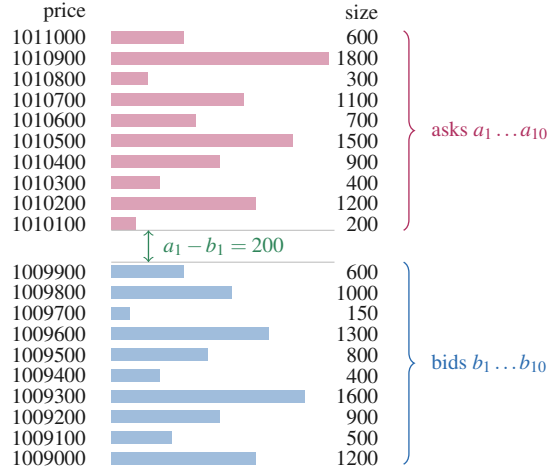


Figure 1. One snapshot, ten levels a side.

A day is N such snapshots,

$$W = \left[\underbrace{\delta \ t_s \ t_n}_{\text{metadata, 3}} \mid \underbrace{a_1 \ u_1 \ b_1 \ w_1 \ \cdots \ a_{10} \ u_{10} \ b_{10} \ w_{10}}_{\text{snapshot, 40}} \right] \in \mathbb{R}^{N \times 43}, \quad (2)$$

where δ is the change in the mid, and t_s and t_n are the seconds and nanoseconds of the event's timestamp, kept apart so that neither loses integer precision. Converting the seconds to nanoseconds and adding the

remainder gives the time of the event,

$$\tau = 10^9 t_s + t_n, \quad (3)$$

which is what places it on the grid in (5).

1.2 Discrete time

The book has a state at nanosecond resolution. It is sampled at 1 ms intervals,

$$t_k = t_0 + k\Delta, \quad k = 0, 1, 2, \dots, \quad \Delta = 1 \text{ ms}, \quad (4)$$

with F_k the book at t_k ,

$$F_k = S_{t_k} \in \mathbb{R}^{10 \times 4}. \quad (5)$$

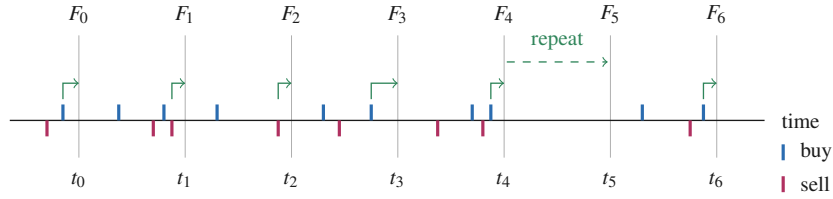


Figure 2. Each frame holds the book as the last event left it.

This is done over the ten level book data, one trading day of one ticker at a time. The corpus covers the S&P 500 from 2022 to 2025, and the first state model runs used GOOG.

The frames are cut into consecutive windows of $L = 512$,

$$F = \begin{bmatrix} F_0 \\ \vdots \\ F_{L-1} \end{bmatrix} \in \mathbb{R}^{512 \times 40}, \quad (6)$$

and the day is these windows end to end,

$$\underbrace{F^{(1)}, F^{(2)}, F^{(3)}, \dots, F^{(M)}}_{\text{one day, } M \approx 45,700}. \quad (7)$$

Given the first n rows of a window, the remaining $L - n$ are generated in one pass. Training draws n from $\{0, \dots, 256\}$, so between 256 and 512 ms is generated.

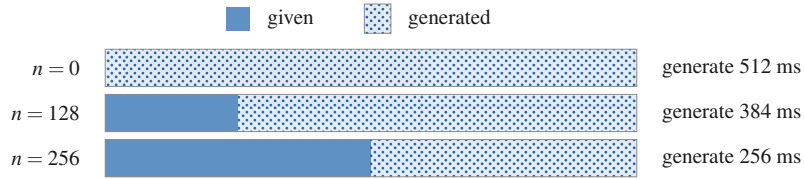


Figure 3. Whatever is given, the rest of the window is generated.

1.3 The chart transform

The book states are encoded into a different but lossless representation. The model reads and writes these coordinates, and the book is recovered by decoding at the end. With the mid

$$m_k = \frac{1}{2}(a_1 + b_1) \quad (8)$$

at frame k , Table 1 gives every entry.

	in words	expression
s	the log spread	$\log(a_1 - b_1)$
d	change in the mid over one frame	$m_k - m_{k-1}$, zero at $k = 0$
g_j	log distance between ask levels j and $j + 1$	$\log(a_{j+1} - a_j)$
h_j	log distance between bid levels j and $j + 1$	$\log(b_j - b_{j+1})$
u_j	depth at ask level j	unchanged
w_j	depth at bid level j	unchanged

Table 1. The entries of a chart frame.

Distances run $j = 1, \dots, 9$ and depths $j = 1, \dots, 10$, and each logarithm takes a floor of 1, the price unit. A snapshot and a window then read

$$C_k = \begin{bmatrix} s & u_1 & d & w_1 \\ g_1 & u_2 & h_1 & w_2 \\ \vdots & \vdots & \vdots & \vdots \\ g_9 & u_{10} & h_9 & w_{10} \end{bmatrix} \in \mathbb{R}^{10 \times 4}, \quad C = \begin{bmatrix} C_0 \\ \vdots \\ C_{L-1} \end{bmatrix} \in \mathbb{R}^{512 \times 40}, \quad (9)$$

the same shapes as S_t and F . Figure 4 marks the three distances on the book.

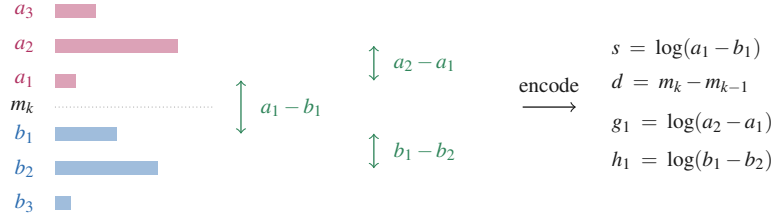


Figure 4. Each price becomes a log distance.

Decoding inverts this,

$$m_k = m_0 + \sum_{i \leq k} d_i, \quad a_1 = m_k + \frac{1}{2}e^s, \quad b_1 = m_k - \frac{1}{2}e^s, \quad (10)$$

$$a_{j+1} = a_j + e^{g_j}, \quad b_{j+1} = b_j - e^{h_j}. \quad (11)$$

with the anchor m_0 , the mid of the first frame, the only absolute price kept. In single precision the round trip returns integer prices exactly.

Why the transform is used.

- Impossible books become impossible. A crossed book, with buys above sells, has no representation in these coordinates. BookDiT reports 54.4 percent of frames crossed when the same model is trained in raw coordinates, against 0.000 in this one [1].
- Fewer digits, and a stationary target. Raw prices are of order 10^6 and drift over months, where their log distances are of order 10 and do not.

1.4 The input tensor

Each row of a window is a frame, the book at one instant, and each column is a channel, a single quantity followed across the whole window. Figure 5 shows the two readings.

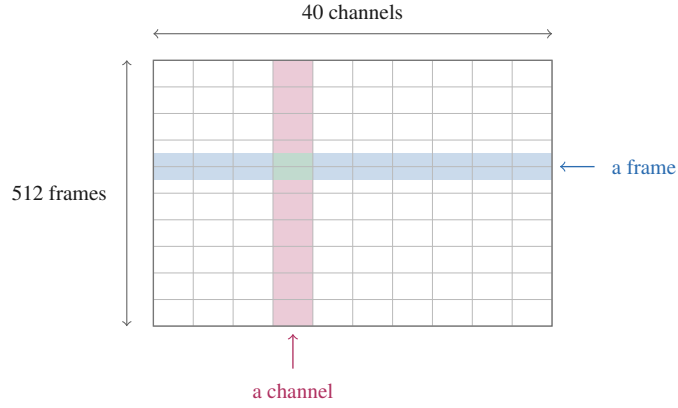


Figure 5. A window read down its columns and across its rows.

The forty channels differ by orders of magnitude, a log spread near five sitting beside a depth in the thousands, so each is standardised separately over the training days,

$$\tilde{C} = \frac{C - \mu}{\sigma}, \quad \mu, \sigma \in \mathbb{R}^{40}, \quad (12)$$

with one mean and one standard deviation per channel. Both are stored inside every checkpoint, so that generation reproduces the scale the model was trained on. Windows are then stacked into batches, and the tensor the network is given is

$$X = \begin{bmatrix} \tilde{C}^{(1)} \\ \vdots \\ \tilde{C}^{(B)} \end{bmatrix} \in \mathbb{R}^{32 \times 512 \times 40}. \quad (13)$$

The logarithm of one plus the count of events in each interval can be appended as a 41st channel, off by default.

quantity	symbol	default
frame interval	Δ	1 ms
levels		10
channels		40
window length	L	512 frames, 512 ms
maximum context	n	256 frames
batch	B	32

Table 2. Defaults.

One training example is half a second of book on an axis of time and an axis of channel, the same object a video model denoises.

2 Pretraining

2.1 The objective

Training consumes the tensor of Section 1.4. One row holds one frame, and its forty entries are the quantities of Table 1 ordered by book level,

$$\left(\underbrace{s \ u_1 \ d \ w_1}_{\text{level 1}} \ \underbrace{g_1 \ u_2 \ h_1 \ w_2}_{\text{level 2}} \ \cdots \ \underbrace{g_9 \ u_{10} \ h_9 \ w_{10}}_{\text{level 10}} \right) \in \mathbb{R}^{40}, \quad (14)$$

where each entry is standardised by (12). A window is 512 consecutive rows and a batch is 32 windows,

$$X \in \mathbb{R}^{32 \times 512 \times 40}, \quad (15)$$

each batch supplying one gradient step. Figure 6 shows a batch as a stack of windows.

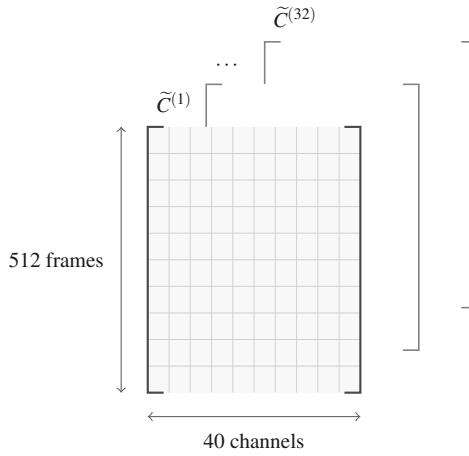


Figure 6. The batch tensor X , thirty two windows of 512 frames by 40 channels.

Each batch comprises thirty two independent draws,

$$X^{(b)} \sim p, \quad \varepsilon^{(b)} \sim \mathcal{N}(0, I), \quad t_b \sim \mathcal{U}[0, 1], \quad n_b \sim \mathcal{U}\{0, \dots, 256\}, \quad (16)$$

where p is the distribution of the windows of Section 1 and the entries of $\varepsilon^{(b)}$ are independent. The window and its noise are linearly interpolated at the level drawn for that slot,

$$x^{(b)} = (1 - t_b)X^{(b)} + t_b \varepsilon^{(b)}, \quad (17)$$

which equals $X^{(b)}$ at $t = 0$ and $\varepsilon^{(b)}$ at $t = 1$.

	$\tilde{s}$	$\tilde{u}_1$	$\tilde{d}$	$\tilde{w}_1$	
X	0.42	-1.13	0.05	0.88	$t = 0$
	0.24	-0.41	-0.12	0.69	$t = 0.25$
	0.06	0.32	-0.29	0.51	$t = 0.5$
	-0.13	1.05	-0.45	0.32	$t = 0.75$
ε	-0.31	1.77	-0.62	0.14	$t = 1$
$\varepsilon - X$	-0.73	2.90	-0.67	-0.74	the target

Table 3. Four of the forty standardised channels of one frame, its noise, and their interpolants.

The interpolant traverses the segment from $X^{(b)}$ to $\varepsilon^{(b)}$ in unit time, and its velocity

$$\frac{dx^{(b)}}{dt} = \varepsilon^{(b)} - X^{(b)} \quad (18)$$

is the regression target.

The network takes $x^{(b)}$ and t_b as arguments and predicts the velocity. The minimiser of the squared error is the conditional mean,

$$v^*(x, t) = \underset{v}{\operatorname{argmin}} \mathbb{E} \left\| (\varepsilon - X) - v(x_t, t) \right\|^2 = \mathbb{E}[\varepsilon - X \mid x_t = x], \quad (19)$$

the average velocity of the interpolants passing through x at time t [3]. Integrating this field carries noise to samples distributed as p , and Section 3 gives the solver.

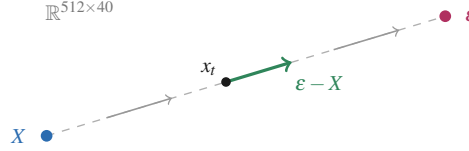


Figure 7. The segment from X to ε in $\mathbb{R}^{512 \times 40}$, and its constant velocity.

The field is modelled by a network v_θ with parameters $\theta \in \mathbb{R}^P$. Writing $\|\cdot\|^2$ for the mean squared entry of its argument, the batch objective is

$$L(\theta) = \frac{1}{32} \sum_{b=1}^{32} \left\| (\varepsilon^{(b)} - X^{(b)}) - v_\theta(x^{(b)}, t_b) \right\|^2. \quad (20)$$

The draws are held fixed, so L depends on θ alone, and it is minimised by descent. The output layer of v_θ is initialised at zero, so v_θ vanishes identically before training. Section 2.2 conditions the model on part of each window, which replaces (20) with the objective actually used.

2.2 Conditioning

Sampling from p produces 512 ms of a book resembling recorded ones but bearing no relation to any particular book. A simulator starts from the book as it stands and asks what follows, so what is required is the distribution of continuations of a given beginning. Each window is therefore drawn with a context length

$$n_b \sim \mathcal{U}\{0, \dots, 256\}, \quad (21)$$

fixing how much of it is given to the network and how much it must produce, and the model samples from

$$p(X_{\geq n_b} \mid X_{< n_b}), \quad (22)$$

where $X_{< n}$ and $X_{\geq n}$ are the first n frames of a window and the remainder.

The given frames are supplied on their own channels beside the interpolant, together with an indicator of which frames they are, giving 81 input channels against the 40 of the unconditional case. Writing n for n_b , the input is

$$\begin{bmatrix} x_0 & X_0 & 0 \\ \vdots & \vdots & \vdots \\ x_{n-1} & X_{n-1} & 0 \\ x_n & 0 & 1 \\ \vdots & \vdots & \vdots \\ x_{511} & 0 & 1 \end{bmatrix} \mapsto \begin{bmatrix} \hat{v}_0 \\ \vdots \\ \hat{v}_{n-1} \\ \hat{v}_n \\ \vdots \\ \hat{v}_{511} \end{bmatrix} \quad (23)$$

from $\mathbb{R}^{512 \times 81}$ to $\mathbb{R}^{512 \times 40}$, the input blocks being the interpolant, the window on the given frames and the indicator, of widths 40, 40 and 1. Here x_k and X_k are the 40 channels of frame k in the interpolant and in the window, and $\hat{v}_k$ is the velocity returned at that row.

The first n_b rows of the output are discarded. The rest are scored against the target and averaged,

$$\ell_b = \frac{1}{512 - n_b} \sum_{k=n_b}^{511} \|\hat{y}_k^{(b)} - (\mathbf{e}_k^{(b)} - X_k^{(b)})\|^2, \quad (24)$$

with the norm the mean squared entry over the forty channels of a row, and the objective is their mean over the batch,

$$L(\theta) = \frac{1}{32} \sum_{b=1}^{32} \ell_b. \quad (25)$$

Dividing by $512 - n_b$ weights every window equally, whatever fraction of it is generated. On a given row the window already appears in the input, so its target follows from the interpolation and it is excluded.

The interpolant is noised at every frame and the given frames are never substituted into it, so training and generation present the same input.

The range of n_b trains one network across every amount of history, and $n_b = 0$ supplies the unconditional case.

2.3 The optimiser

The network defines a parameterised function v_θ with P trainable parameters,

$$\theta = (\theta_1, \dots, \theta_P) \in \mathbb{R}^P.$$

For an input (x, t) , the forward pass produces the prediction $\hat{v}$ of (23), from which the scalar batch loss $L(\theta)$ of (25) is evaluated. The loss depends on the parameters through the intermediate computations of the network,

$$(x, t; \theta) \longrightarrow z_1 \longrightarrow \dots \longrightarrow z_m \longrightarrow \hat{v} \longrightarrow L. \quad (26)$$

The dependence of the loss on each parameter is described by the gradient

$$\nabla_\theta L = \left[\frac{\partial L}{\partial \theta_1}, \dots, \frac{\partial L}{\partial \theta_P} \right] \in \mathbb{R}^P. \quad (27)$$

Each component $\partial L / \partial \theta_i$ gives the local sensitivity of the loss to θ_i , with the remaining parameters held fixed. Backpropagation computes these derivatives efficiently by applying the chain rule backwards through the computational graph. Along a single path

$$\theta_i \rightarrow z_1 \rightarrow \dots \rightarrow z_m \rightarrow L,$$

the corresponding derivative is

$$\frac{\partial L}{\partial \theta_i} = \frac{\partial L}{\partial z_m} \frac{\partial z_m}{\partial z_{m-1}} \dots \frac{\partial z_1}{\partial \theta_i}, \quad (28)$$

with contributions accumulated over all paths connecting θ_i to L . The gradient therefore provides a local direction in parameter space along which the loss changes.

An optimiser converts these gradients into parameter updates. Writing

$$G_{i,t} = \frac{\partial L_t}{\partial \theta_i} \quad (29)$$

for the gradient of parameter θ_i at optimisation step t , gradient descent gives

$$\theta_i^{(t+1)} = \theta_i^{(t)} - \eta G_{i,t}, \quad (30)$$

where $\eta > 0$ is the learning rate. A single learning rate therefore scales all parameter updates, despite potentially substantial differences in gradient scale between parameters and over the course of training.

Adam [2] instead constructs a parameter-wise adaptive update from the recent history of each parameter’s gradients. It maintains exponentially weighted estimates of the gradient and squared gradient,

$$M_{i,t} = \beta_1 M_{i,t-1} + (1 - \beta_1) G_{i,t}, \quad (31)$$

and

$$V_{i,t} = \beta_2 V_{i,t-1} + (1 - \beta_2) G_{i,t}^2. \quad (32)$$

Here, $M_{i,t}$ is a smoothed estimate of the recent gradient, while $V_{i,t}$ is a smoothed estimate of its squared magnitude. Consequently, $\sqrt{V_{i,t}}$ provides an estimate of the recent root-mean-square gradient magnitude. Dividing the first moment by this quantity normalises the update by the characteristic gradient scale of that parameter.

Both moments are initialised at zero, $M_{i,0} = V_{i,0} = 0$, which biases their early values towards zero. Adam removes this initialisation bias using

$$\hat{M}_{i,t} = \frac{M_{i,t}}{1 - \beta_1^t}, \quad \hat{V}_{i,t} = \frac{V_{i,t}}{1 - \beta_2^t}. \quad (33)$$

The factors $1 - \beta_1^t$ and $1 - \beta_2^t$ account for the missing history at early optimisation steps; as t increases, both corrections approach unity.

The complete Adam update is therefore

$$\theta_i^{(t+1)} = \theta_i^{(t)} - \eta \frac{\hat{M}_{i,t}}{\sqrt{\hat{V}_{i,t} + \xi}}, \quad (34)$$

where $\xi > 0$ ensures numerical stability when the estimated gradient magnitude is small. The numerator determines the smoothed descent direction, while the denominator rescales it by the recent gradient magnitude. Adam therefore reduces the dependence of each update on the absolute scale of its parameter’s gradient, yielding a parameter-wise adaptive effective learning rate.

The optimiser parameters used here are summarised in Table 4.

Symbol	Meaning	Value
η	learning rate	5×10^{-4}
β_1	first-moment decay	0.9
β_2	second-moment decay	0.999
ξ	numerical stability constant	10^{-8}

Table 4. Optimiser settings.

Before the Adam moments are formed, the full gradient is clipped to unit norm. When clipping is active, this preserves the gradient direction while bounding its magnitude and limiting the effect of anomalously large gradients. Training is performed for 10^4 optimisation steps.

Backpropagation thus computes the sensitivity of the loss to each network parameter, while Adam converts these gradients and their recent first- and second-moment estimates into parameter updates.

Algorithm 1 Pretraining

1. **repeat**
 2. $X \sim p, \quad \varepsilon \sim \mathcal{N}(0, I), \quad t \sim \mathcal{U}[0, 1], \quad n \sim \mathcal{U}\{0, \dots, 256\}$
 3. $x \leftarrow (1 - t)X + t\varepsilon$
 4. assemble the input of (23) and read off $\hat{v}$
 5. take an Adam step on $\nabla_{\theta} \ell$, the error of (24) over rows n to 511
 6. **until** converged
-

3 Sampling

At inference, sampling begins from a 512×40 tensor of independent standard Gaussian noise,

$$x_1 = \varepsilon, \quad \varepsilon_{ij} \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, 1).$$

The network $v_\theta(x_t, t; c)$ parameterises a time-dependent velocity field over the state space. This field describes how the current state x_t should change at flow time t and defines the ODE

$$\frac{dx_t}{dt} = v_\theta(x_t, t; c),$$

where c denotes the conditioning information of (23), the given frames together with their indicator. Sampling solves this ODE from $t = 1$ to $t = 0$, transporting the initial Gaussian noise towards the learned data distribution.

The flow coordinate t is artificial and is distinct from market time. Market time is represented by the 512 rows of x_t , separated by 1 ms. Each integration step therefore updates the complete 512×40 window jointly rather than generating market frames sequentially. The temporal evolution of the order book is learned as part of the joint structure of the complete window.

For a context length n , the observed prefix $X_{<n}$ is supplied through the conditioning channels of (23), while the remaining $512 - n$ frames are generated. The resulting continuation approximates a sample from

$$X_{\geq n} \sim p(X_{\geq n} | X_{<n}).$$

Using Euler integration with step size h , the state is updated according to

$$x_{t-h} = x_t - h v_\theta(x_t, t; c),$$

with the velocity field re-evaluated after each step. At $t = 0$, the generated rows $X_{\geq n}$ are retained and transformed back to the original order-book representation. Independent noise draws produce different plausible continuations from the same observed context.

When $n = 0$, no market context is supplied and the model generates an unconditional 512 ms window. For $n > 0$, the generated horizon is $512 - n$ milliseconds. The model therefore conditions only on the observed prefix within the current window, while earlier market history is not explicitly available during generation.

References

- [1] **BookDiT**. Chart representation and WanDiT backbone for limit order book diffusion. ICAIF 2026 submission, cited by `src/state`.
- [2] **Adam**. D. P. Kingma, J. Ba. Adam, A Method for Stochastic Optimization. ICLR 2015. [arXiv 1412.6980](https://arxiv.org/abs/1412.6980).
- [3] **Rectified flow**. X. Liu, C. Gong, Q. Liu. Flow Straight and Fast, Learning to Generate and Transfer Data with Rectified Flow. ICLR 2023, UT Austin. [arXiv 2209.03003](https://arxiv.org/abs/2209.03003).